Plano de Refatoração SOLID

CatólicaReserva — Servidor / API REST (Node.js + Express + MySQL)

Disciplina: Projeto e Arquitetura de Software
Integrantes: Gustavo Schmitt e Heric Moreira Ribeiro
Data: outubro de 2026 · apresentação prevista para 06/11

OBJETIVO DESTE DOCUMENTO

Descrever, de forma prática, **quais alterações o código do servidor precisa** para atender aos cinco princípios SOLID e aos 13 critérios de avaliação da atividade, mantendo a consistência com os diagramas C4 (Container e Componentes) entregues na N1. Para cada princípio, o documento mostra o que existe hoje, por que isso é um problema e o que deve ser feito.

Sumário

1. O que é SOLID
2. Diagnóstico do código atual
3. Alterações por princípio (S, O, L, I, D)
4. Estrutura de pastas proposta
5. Atendimento aos 13 critérios da atividade
6. Ordem sugerida de implementação
7. Cuidados e observações

1. O que é SOLID

SOLID é um conjunto de cinco princípios de design orientado a objetos. Eles servem para deixar o código mais fácil de entender, testar e evoluir sem quebrar o que já funciona. A atividade pede que o código evite três sintomas:

- **Classes que fazem "de tudo um pouco"**: qualquer mudança obriga a mexer em um arquivo que não tem relação com ela.
- **if/else extensos que travam a evolução**: toda regra nova exige editar o mesmo bloco de código.
- **Interfaces gigantescas com métodos que ninguém usa**: quem implementa é obrigado a escrever código inútil.

Letra	Princípio	Ideia central
S	Single Responsibility Responsabilidade Única	Uma classe deve ter um único motivo para mudar.
O	Open/Closed Aberto/Fechado	Aberto para extensão, fechado para modificação: comportamento novo entra com código novo.
L	Liskov Substitution Substituição de Liskov	Quem cumpre um contrato pode substituir outro que cumpre o mesmo contrato sem quebrar nada.

I	Interface Segregation Segregação de Interfaces	Ninguém deve depender de métodos que não usa: interfaces pequenas e específicas.
D	Dependency Inversion Inversão de Dependência	Classes de alto nível dependem de abstrações, não de implementações concretas.

2. Diagnóstico do código atual

O projeto parte de uma base boa: a divisão **Controller → Service → Repository** já existe, cada componente do diagrama C4 Nível 3 tem um arquivo correspondente e não há classes gigantes. Os problemas estão concentrados em três pontos:

PRINCIPAIS PROBLEMAS

- **Dependências fixas:** cada service faz `require` direto do repositório concreto, e cada repositório faz `require` direto do pool MySQL. Não há injeção de dependência nem um ponto central de montagem.
- **Regra de negócio misturada com infraestrutura:** o `usuarioService` usa `bcrypt` e `jsonwebtoken` diretamente, e todos os services criam erros com status HTTP (`erro.status = 409`).
- **Validações em sequência de if:** cada regra nova de reserva (limite de horário, antecedência, previstas no README) obriga a editar `reservaService.criar` .

Arquivo	O que acontece hoje	Princípio afetado
`services/usuarioService.js`	Valida dados, gera hash (bcrypt), gera token (JWT) e monta a resposta.	S, D
`middlewares/auth.js`	Repete a lógica de JWT e o segredo `JWT_SECRET` já usados no service.	S
`services/reservaService.js`	Quatro validações em sequência de `if` , mais a regra de permissão de cancelamento fixa no código.	O, S
Todos os services	Criam erros com status HTTP e usam nomes de colunas do banco (`r.hora_fim` , `reserva.usuario_id`).	S, D
Todos os repositories	Importam o pool MySQL diretamente (`require('../config/db')`).	D
`diagrama_classes.puml`	Prevê `ReservaRepository.remover(id)` e `cancelar(id)` , que não existem no código.	L, consistência C4

3. Alterações por princípio

S Responsabilidade Única

HOJE

O `usuarioService` tem quatro motivos para mudar: regra de cadastro, algoritmo de hash, tecnologia de token e formato da resposta. Se o JWT for trocado, a regra de cadastro precisa ser editada.

```
// services/usuarioService.js (atual)
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
...
const senhaHash = await bcrypt.hash(senha, 10);
const token = jwt.sign({ id, nome, tipo }, JWT_SECRET, { expiresIn: '8h' });
return { token, usuario: { id, nome, email, tipo } };
```

ALTERAÇÕES NECESSÁRIAS

- Criar `infra/seguranca/BcryptGeradorHash.js` com `gerar(senha)` e `comparar(senha, hash)`.
- Criar `infra/seguranca/JwtServicoToken.js` com `gerar(dados)` e `verificar(token)`. O segredo passa a ser lido em um único lugar.
- O middleware de autenticação passa a usar o `verificar()` dessa classe, eliminando a duplicação.
- Criar erros de domínio em `dominio/erros.js` (`DadosInvalidosError`, `NaoEncontradoError`, `ConflitoError`, `AcessoNegadoError`, `CredenciaisInvalidasError`). Só o `errorHandler` traduz cada um para o status HTTP.
- Mover a formatação do JSON de resposta para a camada HTTP (apresentadores), tirando essa tarefa dos services.

```
// services/UsuarioService.js (proposto)
class UsuarioService {
  constructor({ usuarioRepository, geradorHash, geradorToken }) { ... }

  async autenticar({ email, senha }) {
    const usuario = await this.usuarioRepository.buscarPorEmail(email);
    if (!usuario || !(await this.geradorHash.comparar(senha, usuario.senhaHash))) {
      throw new CredenciaisInvalidasError('E-mail ou senha inválidos.');
    }
    const token = this.geradorToken.gerar({ id: usuario.id, nome: usuario.nome, tipo: usuario.tipo });
    return { token, usuario };
  }
}
```

Critérios atendidos: 4, 6 (e contribui para o 2).

O Aberto/Fechado

HOJE

A criação de reserva é uma sequência de `if`. Cada regra nova prevista no README (limite de horário, antecedência) exige editar essa função, que é o núcleo do sistema.

```
// services/reservaService.js (atual)
if (!salaId || !data || !horaInicio || !horaFim) { ... }  // campos obrigatórios
if (horaFim <= horaInicio) { ... }                        // horário válido
if (!sala) { ... }                                         // sala existe
if (await existeConflito(...)) { ... }                     // conflito de horário
```

ALTERAÇÕES NECESSÁRIAS

- Transformar cada validação em uma classe com o método `validar(reserva)`, dentro de `dominio/regras/reserva/`: `CamposObrigatorios`, `HorarioValido`, `SalaDeveExistir`, `SemConflitoDeHorario`.
- Fazer o mesmo com a permissão de cancelamento: `dominio/regras/cancelamento/ApenasDonoOuAdmin.js`.
- O `ReservaService` recebe as listas de regras pelo construtor e apenas as executa em ordem.
- Uma regra nova (por exemplo, `LimiteDeDuracao`) passa a ser uma classe nova registrada no container, sem alterar o service.
- No `errorHandler`, usar uma tabela "tipo de erro → status HTTP": um erro novo é uma linha a mais, sem novos `if`.

```
// services/ReservaService.js (proposto)
async criar(dados) {
  const reserva = new Reserva(dados);
  for (const regra of this.regrasDeCriacao) {
    await regra.validar(reserva);   // cada regra lança um erro de domínio se falhar
  }
  return this.reservaRepository.salvar(reserva);
}

// Regra nova: só uma classe nova, o service não muda
class LimiteDeDuracao {
  async validar(reserva) { /* lança DadosInvalidosError se passar de 2h */ }
}
```

Critérios atendidos: 7, 8, 6.

L Substituição de Liskov

HOJE

Como ainda não há interfaces nem herança, o princípio é atendido apenas por falta de uso. A inconsistência é o diagrama de classes, que promete `remover(id)` e `cancelar(id)`, métodos que não existem no código.

ALTERAÇÕES NECESSÁRIAS

- Ao criar os contratos (letra I), garantir que **toda** implementação tenha **todos** os métodos, sem métodos vazios, sem retornos `null` forçados e sem lançar "não implementado".
- Criar repositórios em memória para os testes. Se os services funcionam igual com o MySQL e com a versão em memória, a substituição está comprovada.
- Criar um teste automatizado que verifica se cada implementação cumpre seu contrato inteiro.
- Corrigir o diagrama de classes para refletir os métodos reais (`atualizarStatus` , `cancelar(id, usuario)`).

Critérios atendidos: 9, 11.

I Segregação de Interfaces

HOJE

Não existe nenhuma interface definida. O critério 10 não é atendido.

ALTERAÇÕES NECESSÁRIAS

Criar contratos pequenos em `dominio/contratos/` , cada um para um papel específico:

Contrato	Métodos	Quem usa	Quem implementa
`ReservaRepository`	salvar, buscarPorId, listarPorUsuario, listarTodas, atualizarStatus	ReservaService	MySqlReservaRepository
`AgendaDeReservas`	buscarAtivasPorSalaEData	Regra de conflito	MySqlReservaRepository
`SalaRepository`	listar, buscarPorId, salvar	SalaService, regra SalaDeveExistir	MySqlSalaRepository
`UsuarioRepository`	buscarPorEmail, salvar	UsuarioService	MySqlUsuarioRepository
`GeradorHash`	gerar, comparar	UsuarioService	BcryptGeradorHash
`GeradorToken`	gerar	UsuarioService	JwtServicoToken
`VerificadorToken`	verificar	Middleware de autenticação	JwtServicoToken
`RegraReserva`	validar(reserva)	ReservaService	As 4 regras de criação
`RegraCancelamento`	validar(reserva, usuario)	ReservaService	ApenasDonoOuAdmin

- A regra de conflito depende só de `AgendaDeReservas` (1 método), e não do repositório inteiro.
- Gerar e verificar token são contratos separados: o middleware nunca gera token, e o service nunca verifica.
- Métodos que ninguém usa saem do contrato (por exemplo, `usuarioRepository.buscarPorId` , que hoje não é chamado).

JAVASCRIPT NÃO TEM INTERFACE

Diferente de Java ou C#, JavaScript não tem a palavra-chave `interface`. A solução proposta é um pequeno objeto `Contrato` que guarda o nome da interface e a lista de métodos obrigatórios. Cada service chama `Contrato.verificar(dependencia)` no construtor: se faltar algum método, o servidor nem sobe e mostra quais métodos estão faltando.

```
// dominio/contratos/SalaRepository.js
module.exports = new Contrato('SalaRepository', ['listar', 'buscarPorId', 'salvar']);
```

Critérios atendidos: 10, 11.

D Inversão de Dependência

HOJE

Este é o ponto mais fraco do projeto. Os services estão "soldados" às implementações concretas:

```
// services/reservaService.js (atual)
const reservaRepository = require('../repositories/reservaRepository');

// repositories/reservaRepository.js (atual)
const pool = require('../config/db');   // MySQL direto
```

Consequências: não é possível testar a regra de conflito sem um MySQL rodando, trocar o banco exigiria mexer em todos os services, e não existe um lugar que mostre claramente "quem usa quem", como o diagrama C4 mostra.

ALTERAÇÕES NECESSÁRIAS

- Transformar services, repositórios e controllers em classes que recebem as dependências **pelo construtor**.
- Os repositórios recebem o `pool` no construtor, em vez de importá-lo.
- Criar `src/container.js` (Composition Root): o **único** arquivo que faz `new` das classes concretas e monta tudo.
- Criar `config/ambiente.js` para ler o `.env` em um único lugar.
- O `server.js` passa a chamar o container antes de subir a API.

```
// src/container.js (proposto)
function criarContainer(config) {
  const pool = criarPool(config.banco);
  const reservaRepository = new MySqlReservaRepository(pool);
  const salaRepository = new MySqlSalaRepository(pool);

  const reservaService = new ReservaService({
    reservaRepository,
    regrasDeCriacao: [
      new CamposObrigatorios(),
      new HorarioValido(),
      new SalaDeveExistir({ salaRepository }),
      new SemConflitoDeHorario({ agendaDeReservas: reservaRepository }),
    ],
    regrasDeCancelamento: [new ApenasDonoOuAdmin()],
  });
  ...
}
```

Critérios atendidos: 2, 3, 12, 13.

4. Estrutura de pastas proposta

A regra de ouro é que as dependências apontam sempre **para dentro**: a camada HTTP usa os services, os services usam o domínio, e a infraestrutura implementa os contratos do domínio. O domínio não depende de ninguém.

```
http  →  services  →  dominio  ←  infra
```

```
server/src/
├── config/
│   └── ambiente.js            lê o .env em um único lugar
├── dominio/                   núcleo: não importa Express, MySQL, bcrypt nem JWT
│   ├── entidades/             Reserva, Sala, Usuario
│   ├── contratos/             interfaces (Contrato.js + 9 contratos)
│   ├── regras/
│   │   ├── reserva/           4 regras de criação (uma classe cada)
│   │   └── cancelamento/      ApenasDonoOuAdmin
│   └── erros.js               erros de domínio (sem status HTTP)
├── services/                  ReservaService, SalaService, UsuarioService
├── infra/
│   ├── mysql/                 conexao.js + 3 repositórios MySql*Repository
│   └── seguranca/             BcryptGeradorHash, JwtServicoToken
├── http/
│   ├── controllers/           AuthController, SalaController, ReservaController
│   ├── rotas/                 definição das rotas
│   ├── middlewares/           autenticação e errorHandler
│   └── apresentadores/        formatação do JSON de resposta
├── container.js               monta e injeta todas as dependências
├── app.js                     configuração do Express
└── server.js                  ponto de entrada
server/test/                   testes automatizados com repositórios em memória
```

Correspondência com o diagrama C4 Nível 3

Componente no C4 (N1)	Onde fica no código proposto
ReservaController, SalaController, AuthController	`http/controllers/`
ReservaService, SalaService, UsuarioService	`services/`
ReservaRepository, SalaRepository, UsuarioRepository	Contratos em `dominio/contratos/` e implementações MySQL em `infra/mysql/`
Novos: Regras, Contratos, BcryptGeradorHash, JwtServicoToken, Container	Devem ser **adicionados ao diagrama** Nível 3 para manter a consistência

5. Atendimento aos 13 critérios da atividade

#	Critério	Hoje	Alteração que resolve
1	Componente do C4 Nível 3 ↔ módulo no código	**Atende**	Atualizar o diagrama com os componentes novos (regras, contratos, container).
2	Domínio isolado da infraestrutura	**Não**	Pasta `dominio/` sem bibliotecas externas; bcrypt, JWT e SQL em `infra/` ; erros sem status HTTP.
3	Relações do C4 = injeções no código	**Parcial**	Injeção pelo construtor e `container.js` reproduzindo as setas do diagrama.
4	Uma responsabilidade por classe	**Parcial**	Separar hash, token, formatação do JSON e tradução de erros (letra S).
5	Sem "God Classes"	**Atende**	Manter as classes pequenas.
6	Validação, notificação e persistência delegadas	**Parcial**	Regras em classes próprias; persistência já está nos repositórios.
7	Comportamento novo = classe nova	**Não**	Lista de regras injetada no ReservaService (letra O).
8	Requisito novo não edita o núcleo	**Não**	Mesma alteração do item 7, mais a regra de cancelamento plugável.
9	Sem NotImplemented nas implementações	**Sem interfaces**	Garantir com teste de contratos depois de criar as interfaces.
10	Interfaces coesas	**Não**	Nove contratos pequenos, de 1 a 5 métodos (letra I).
11	Nenhum método vazio forçado	**Sem interfaces**	Mesmo cuidado do item 9.
12	Service depende de interface, não de classe concreta	**Não**	Services recebem contratos pelo construtor (letra D).
13	Dependências montadas no ponto de entrada	**Não**	`container.js` chamado pelo `server.js` .

6. Ordem sugerida de implementação

A ordem abaixo permite que o sistema continue funcionando ao fim de cada etapa e que cada mudança seja testada antes da próxima.

Etapa	O que fazer	Detalhes	Princípios
1	Configuração e erros de domínio	Criar `config/ambiente.js` e `dominio/erros.js`; ajustar o `errorHandler` para traduzir os erros.	S
2	Entidades	Criar `Reserva`, `Sala` e `Usuario`; os repositórios passam a converter linhas do banco em entidades.	S, D
3	Contratos	Criar `Contrato.js` e os nove contratos.	I, L
4	Infraestrutura	Repositórios MySQL como classes que recebem o pool; `BcryptGeradorHash`; `JwtServicoToken`.	S, D
5	Regras plugáveis	Extrair as quatro regras de reserva e a regra de cancelamento para classes.	O
6	Services e controllers com injeção	Construtores recebendo dependências; apresentadores para o JSON.	S, D
7	Container	Criar `container.js` e ajustar `app.js` e `server.js`.	D
8	Testes	Repositórios em memória; testes das regras, dos services e dos contratos (`npm test`, com o test runner nativo do Node).	L, D
9	Documentação	Atualizar o diagrama C4 Nível 3, o diagrama de classes e o README.	Consistência C4

7. Cuidados e observações

MANTER O CONTRATO DA API

O cliente React usa os campos `sala_nome`, `hora_inicio` e `hora_fim` nas respostas de reserva. As rotas, os status HTTP, as mensagens de erro e os nomes dos campos do JSON devem continuar iguais. Os apresentadores da camada HTTP são o lugar certo para manter esse formato. Assim, o cliente não precisa ser alterado.

BUG ENCONTRADO DURANTE A ANÁLISE

O formulário envia as horas como `"11:00"` e o banco devolve `"11:00:00"`. Na comparação de texto, `"11:00"` é menor que `"11:00:00"`, então uma reserva das 11h às 12h é recusada como conflito com outra das 10h às 11h, mesmo sem sobreposição. Na refatoração, a entidade `Reserva` deve padronizar as horas em `HH:MM:SS`, e um teste deve cobrir esse caso.

COMO VALIDAR NO FINAL

- Rodar `npm test` (não precisa do banco).
- Subir a API e repetir o fluxo pelo cliente: cadastro, login, nova reserva, reserva em conflito, reserva encostada (que deve ser aceita) e cancelamento.
- Conferir que um aluno não consegue cancelar a reserva de outra pessoa e que só o administrador cadastra salas.